



xxxx前端架构设计

前端开发组

xxxx有限公司

2018-12-11

写在前面

Web应用复杂度的增加，特别是单页面应用的风靡。组件化，工程化，自动化成了前端发展的趋势。每个前端团队都在打造自己的前端开发体系，这通常是一个东拼西凑，逐渐磨合的过程，在技术发展日新月异的今天，这样的过程真的是不可抽象和复制的么？本文希望能够通过系统的拆解前端开发体系为大家提供体系设计思路参考。



架构设计

1. 前后端分离
2. SPA
3. 设计模式
4. 项目架构
5. 编码规范
6. 构建部署
7. 常见问题



/01

前后端分离

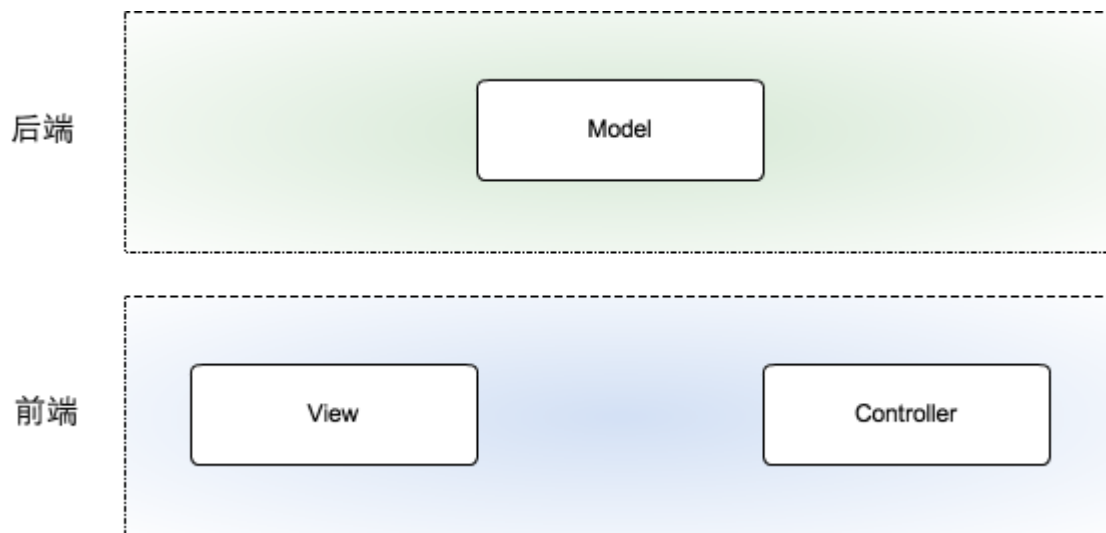


为什么要前后端分离

- 前后端职责不清：在业务逻辑复杂的系统里，我们最怕维护前后端混杂在一起的代码，因为没有约束，M-V-C每一层都可能出现别的层的代码，日积月累，完全没有维护性可言。
- 开发效率低下：前端强依赖后端，需要配置后端开发环境，开发模式依然是，前端写好静态demo，后端翻译成VM模版
- 前端发挥局限：性能优化如果只在前端做空间非常有限，于是我们经常需要后端合作才能碰撞出火花，但由于后端框架限制，我们很难使用Comet、Bigpipe等技术方案来优化性能

如何做到前后端分离

- 后端专注于：后端控制层（ Restful API ） & 服务层 & 数据访问层
- 前端专注于：前端控制层（ Nodejs ） & 视图层



前后端分离注意事项

- 跨域
 - 同站点部署，不存在资源跨域，开发阶段本地开启Nodejs 代理转发 API 请求
 - 不同站点部署，开发阶段本地开启 Nodejs 代理转发 API 请求，部署上线阶段服务端和前端通过 CORS 实现资源共享
- 请求
 - 后端接口返回不再是 ModelAndView，取而代之的是 Model 数据
 - 前端和后端通过 Restfull API 接口数据传输，流转控制下放到前端处理
 - 需求阶段前后端需约定 API 接口规范，数据格式等
 - 开发阶段前端通过 Mock 模拟数据请求，独立后端开发

/02

SPA



什么是SPA

单页Web应用（single page web application，SPA），就是只有一张Web页面的应用。单页应用程序（SPA）是加载单个HTML 页面并在用户与应用程序交互时动态更新该页面的Web应用程序。浏览器一开始会加载必需的HTML、CSS和JavaScript，所有的操作都在这张页面上完成，都由JavaScript来控制。因此，对单页应用来说模块化的开发和设计显得相当重要。

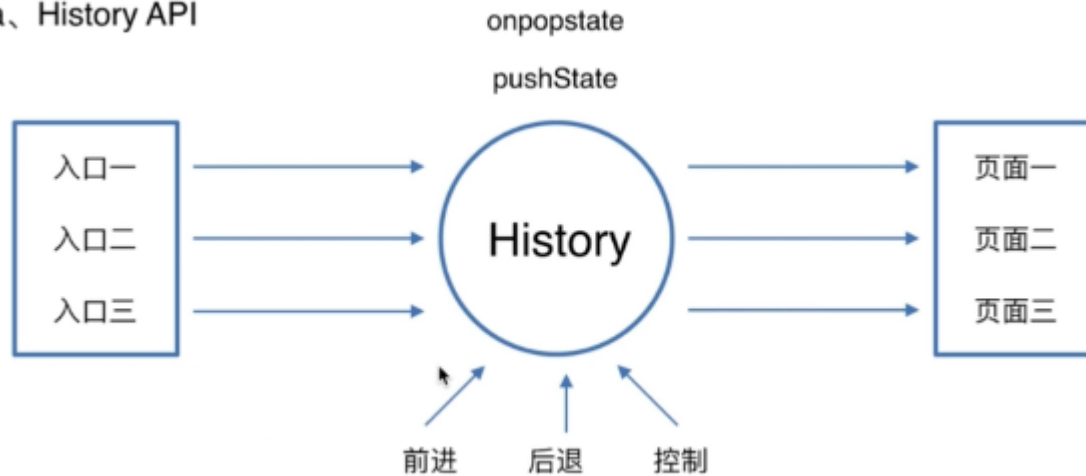
为什么要使用SPA

- 速度：更好的用户体验，让用户在 Web App 感受 Native App 的速度和流畅
- MVVM：经典 MVVM 开发模式，前后端各负其责
- AJAX：重前端，业务逻辑全部在本地操作，数据都需要通过 AJAX 同步、提交
- 路由：在URL中采用#号来作为当前视图的地址,改变#号后的参数，页面并不会重载
- 减轻服务器压力：逻辑跳转在浏览器侧操作，不用频繁请求服务器
- Prerender预渲染

SPA实现原理

- History API
 - pushState : 创建历史记录，追加 url
 - onpopstate : 响应浏览器的前进和后退

a、History API



SPA实现原理

- Hash
 - Location.hash：改变当前页hash值，进而改变Url
 - hashchange：响应hash变化，控制页面切换

b、Hash



/03

设计模式



MVVM

MVVM模式的全称为 Model View ViewModel。它的侧重点在于用事件驱动的 UI 平台的开发。即View的变化会自动更新到 ViewModel，而 ViewModel 的变化也会自动同步到 View 上显示。MVVM 的本质是通过数据绑定链接 View 和 Model，让数据的变化自动映射为视图的更新。

模块化

当一个项目开发的越来越复杂的时候，会遇到一些问题，比如：

- 命名冲突：当项目由团队进行协作开发的时候，不同开发人员的变量和函数命名可能相同；即使是一个开发，当开发周期比较长的时候，也有可能会忘记之前使用了什么变量，从而导致重复命名，导致命名冲突。
- 文件依赖：代码重用时，引入 js 文件的数目可能少了，或者引入的顺序不对，比如使用 bootstrap 的时候，需要引入 jQuery，并且 jQuery 的文件必须要比 bootstrap 的 js 文件先引入。

当使用模块化开发的时候可以避免以上的问题，并且让开发的效率变高，以及方便后期的维护：

- 提升开发效率：代码方便重用，别人开发的模块直接拿过来就可以使用，不需要重复开发类似的功能。
- 方便后期维护：代码方便重用，别人开发的模块直接拿过来就可以使用，不需要重复开发类似的功能。

技术栈

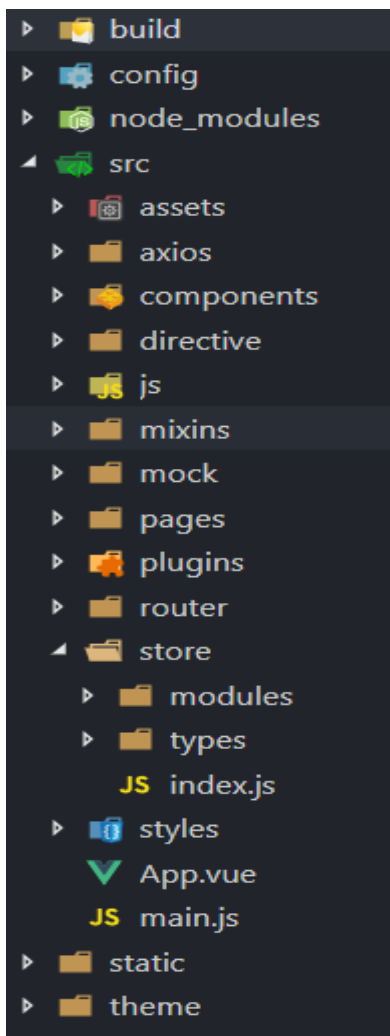
- Vuejs：作为 MVVM 框架中类型的佼佼者，Vuejs 相比于 Angular、React 拥有更好的性能，更易上手的 API，拥抱经典的 Web 技术
- Vue Router：Vue.js 官方的路由管理器。它和 Vue.js 的核心深度集成，让构建单页面应用变得易如反掌
- Vuex：Vuex 是一个专为 Vue.js 应用程序开发的状态管理模式。它采用集中式存储管理应用的所有组件的状态，并以相应的规则保证状态以一种可预测的方式发生变化。Vuex 也集成到 Vue 的官方调试工具 devtools extension，提供了诸如零配置的 time-travel 调试、状态快照导入导出等高级调试功能
- Axios：基于promise用于浏览器和node.js的http客户端
- Element UI + Mint UI：一套为开发者、设计师和产品经理准备的基于 Vue 2.0 的组件库
- ES6：紧跟时代潮流不解释
- Webpack：打包构建一条龙

/04

项目架构

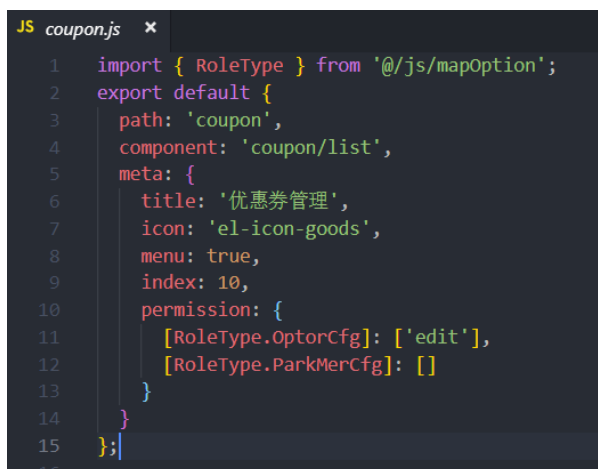
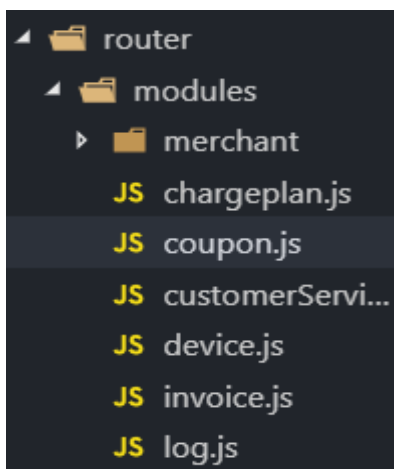


项目结构



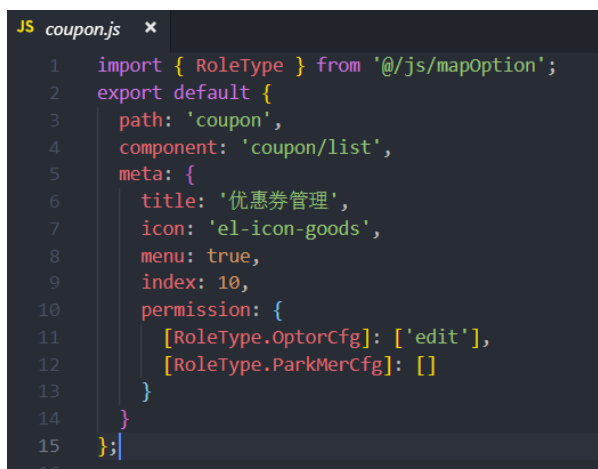
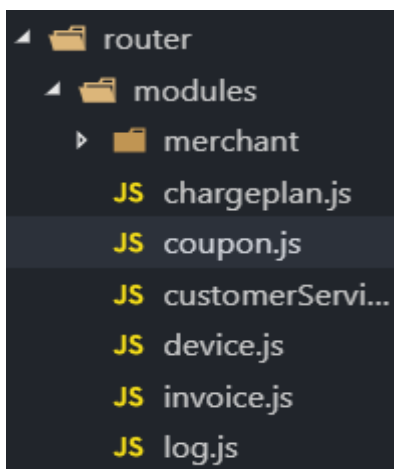
- build : 存放 Webpack打包配置文件
- config : 运行环境配置 , dev、sit、uat、prod
- assets : 静态资源存放目录
- axios : ajax 请求 API 目录
- components : 公共组件存放目录
- directive : 自定义指令
- js : 公共配置
- mixins : 全局方法
- mock : 模拟数据接口目录 , 用于提供模拟数据 , 方便测试
- pages : 业务页面
- plugins : Vue 插件存放目录 , 主要存放一些比较通用的插件
- router : Vue 路由文件存放目录
- store : 全局状态存储
- styles : 全局样式
- static : 静态文件存放目录 , 比如 pdf、doc 等等
- theme : 主题存放目录

路由 – 配置说明



- redirect：路由重定向url
- title：路由名称
- icon：路由对应的图标（通常用于侧边栏菜单配置）
- menu：是否显示在侧边栏（通常用于后管侧边栏菜单配置）
- index：索引，影响该路由在最终路由表中的顺序
- noDropDown：是否显示展开箭头（侧边栏菜单）
- noredirect：是否支持重定向
- permission：角色权限

路由 – 分级配置

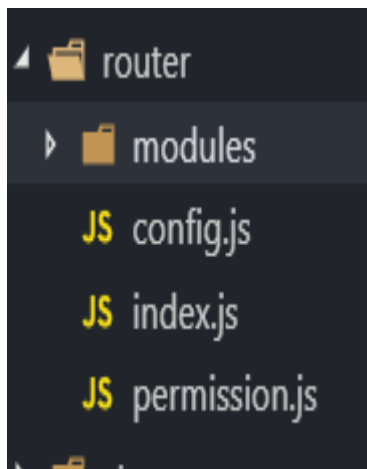


随着项目的迭代，路由文件无可避免的会越来越大，为了便于管理，更好的维护路由表，将路由表拆分为主表和子表：

- 主表：配置公共路由，比如 404、401、关于等等非业务相关型路由
- 子表：通过建立 modules 文件夹，分离路由表，modules 文件夹存放各个业务模块的子路由，相互之间无依赖，支持热插拔

项目构建阶段，配置脚本会动态扫描 modules 目录下各个子路由表，最终将子表和主表组合为一张大的路由表，进而实现拆分维护的目的

路由 – 登陆校验



```
router.beforeEach((to, from, next) => {  
  NProgress.start();  
  // 初始化用户信息 (详情 /store/modules/user)  
  store.dispatch(INIT_USER);  
  let title = to.meta.title || to.params.title;  
  if (!title) {  
    let match = to.matched[to.matched.length - 1];  
    match && match.parent && (title = match.parent.meta.title);  
  }  
  document.title = title || '';  
  router.app.$store = router.app.$store || store;  
  permission(router.app, to, from)  
    .then(path => next(path))  
    .catch(errPath => {  
      next(errPath);  
      NProgress.done(); // 在hash模式下 改变手动改变hash 重定向回页  
    });  
});
```

配置路由前置拦截监听器 “beforeEach”，判断本地是否存在登陆记录，不存在跳转登陆路由，存在继续放行

路由 – 权限配置

- 配置当前页面所有权限（页面级配置，此处配置的“view”需和页面相对应）

```
{
  meta: {
    title: '停车场详情',
    permission: ['200', '100']    //这里配置角色数组, 拥有当前页面所有权限
  }
}
```

- 配置当前页面操作权限（按钮级配置，此处配置的“view”需和页面相对应）

```
{
  meta: {
    title: '停车场详情',
    permission: {
      '101': [],                //例: 空数组拥有当前页面所有权限
      '100': ['view', 'edit'], //例: 100 角色既可查看又可编辑
      '200': ['view'],         //例: 200 仅能查看不能编辑, view 可自己定义
    }
  }
}
```

路由 – 权限集成

路由配置文件中通过 “permission” 配置权限过后，需要构建脚本提取静态路由和动态路由（配置了可访问权限的路由），用于区分不同的登陆角色所看到的页面不同。

用户登陆后开始根据用户登陆角色过滤可访问的路由表

```
/**
 * 递归过滤异步路由表，返回符合用户角色权限的路由表
 * @param asyncRoutes
 * @param roles 用户角色 [103,104]
 * @param filter 过滤函数, function
 */
function filterAsyncRouter(asyncRoutes, roles, filter) {
  const accessedRouters = a any routes.filter(route => {
    if ((route.meta || {}).constant) return true;
    if (hasPermission(roles, route, filter)) {
      if (route.children && route.children.length) {
        route.children = filterAsyncRouter(route.children, roles, filter);
      }
      return true;
    }
    return false;
  });
  return accessedRouters;
}
```

```
/**
 * 通过meta.role判断是否与当前用户权限匹配
 * @param roles 用户角色 [103,104]
 * @param route
 * @param filter 过滤函数, function
 */
function hasPermission(roles, route, filter) {
  if (roles && roles.length > 0) {
    if ((route.meta || {}).constant) return true;
    let permissionList = (route.meta || {}).permission;
    if ((route.path === '/' || route.path === '/home') && !permissionList) {
      // 首页路由，不配置权限时，默认所有角色可以访问，配置权限时根据权限配置访问
      return true;
    } else if (permissionList) {
      return (Array.isArray(permissionList) ?
        permissionList :
        Object.keys(permissionList)
      ).some(permission =>
        roles.some(
          role =>
            String(role) === String(permission) &&
            (filter ? filter(permission, role) : true)
        )
      );
    }
  }
  return false;
}
```

路由 – 权限集成

将根据用户角色过滤后的路由表加入到主路由表中去，完成链路跳转

```
vue.$store
  .dispatch(GET_ROLE_INFO)
  .then(roles => {
    vue.$store
      .dispatch(GENERATE_ROUTES, {
        roles
      })
      .then(accessedRoutes => {
        // 生成可访问的路由表
        vue.$router.addRoutes(accessedRoutes); // 动态添加可访问路由表
        resolve({ ...to }); // hack方法 确保addRoutes已完成
      });
  })
})
```

路由 – 页面权限使用

路由配置权限仅仅完成了页面可访问性控制，还需要做颗粒度（v-permission）

- 单权限配置（使用v-if：v-if="\$permission('edit')”）

```
<el-button type="primary"
  v-permission="'edit'"
  @click="$refs.basicDialog.open({merCode})">
  <i class="iconfont icon-xinjian- font-size-add" /> 新增
</el-button>
```

- 批量权限配置（使用v-if：v-if="\$permission(['edit', 'view'])”）

```
<el-button type="primary"
  v-permission="['edit', 'view']"
  @click="$refs.basicDialog.open({merCode})">
  <i class="iconfont icon-xinjian- font-size-add" /> 新增
</el-button>
```

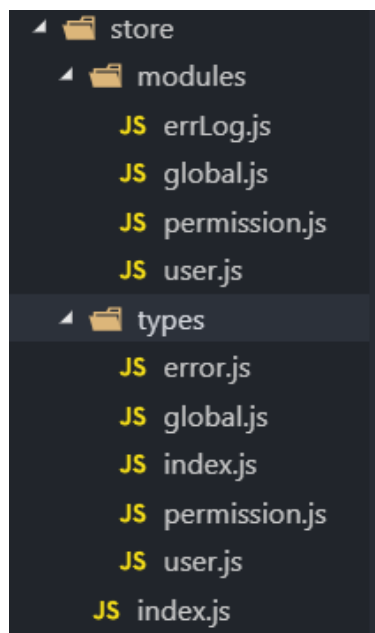
路由 – 懒加载

当打包构建应用时，Javascript 包会变得非常大，影响页面加载。如果我们能把不同路由对应的组件分割成不同的代码块，然后当路由被访问的时候才加载对应组件，这样就更加高效了。

结合 Vue 的异步组件和 Webpack 的代码分割功能，轻松实现路由组件的懒加载。

```
const Foo = () => import('./Foo.vue')
const router = new VueRouter({
  routes: [
    { path: '/foo', component: Foo }
  ]
})
```

状态管理 – 模块配置



```
import {
  LOGOUT,
  INIT_USER,
  UPDATE_USER,
  UPDATE_ROLES,
  GET_ROLE_INFO,
  DEL_VISITED_VIEWS,
  RESET_ROUTERS,
  INIT_SOCKET
} from '@store/types';
const user = {
  state: { ...
  },
  getters: { ...
  },
  mutations: { ...
  },
  actions: { ...
  }
};
export default user;
```

```
export const LOGOUT = 'logout'; /**
 * 初始化用户信息
 */
export const INIT_USER = 'initUser';
/**
 * 更新用户信息
 */
export const UPDATE_USER = 'updateUser';
/**
 * 更新角色信息
 */
export const UPDATE_ROLES = 'updateRoles';
/**
 * 获取角色信息
 */
export const GET_ROLE_INFO = 'getRoleInfo';
```

状态管理按场景区分为用户模块、全局配置模块、权限管理模块（动态路由提取）、错误日志模块。

状态内部操作事件处理统一使用常量，避免出现“神秘的数”。

外部操作状态内部数据，统一使用 actions 作为事件分发，actions 可以更好的支持异步操作，对于大型项目来讲，可以提供链式的数据管理，便于开发过程中错误追踪。

Ajax 请求配置

Ajax 数据请求分为 url 和 请求配置

- index.js：配置 axios 请求拦截、响应拦截，携带校验信息等

```
// axios.defaults.withCredentials = true; // 当时并发的http请求
!$globalConfig.debug && (axios.defaults.baseURL = Url.baseURL); //debug模式用 proxyTable 实现跨域请求
axios.defaults.timeout = 30000;
axios.defaults.headers.post['Content-Type'] = 'application/json;charset=utf-8';
let source = axios.CancelToken.source();
// token 不携带白名单
let whitelist = ['v1/login/operatorLogin'];
axios.interceptors.request.use(
  config => { ...
},
  err => { ...
});

//http response 拦截器
axios.interceptors.response.use(
  response => { ...
},
  error => { ...
});
```

- url.js：配置 axios 请求的基准 Url

```
//
const Config = {
  prod: 'http://www.example.com', //Production
  uat: 'http://uat.example.com', //UAT
  sit: 'http://sit.example.com', //SIT
  dev: 'http://dev.example.com' //DEV
};

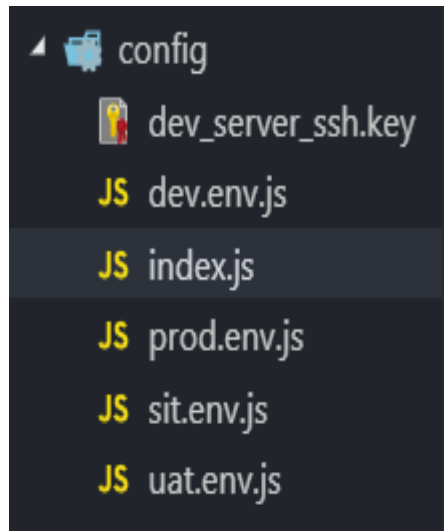
const REQUEST_URL = location.origin || Config[process.env.ENV_CONFIG]; //URL 地址暂时设置为访问地址

const Url = {
  baseUrl: `${REQUEST_URL}/api`,

  /**资源路径 */
  asserts: `${REQUEST_URL}/asserts`
};
export default Url;
```

Ajax 请求代理

前后端分离其中一个问题便是资源跨域，本地开发阶段可以通过 Webpack 配置代理，转发请求



```
dev: {
  env: require('./dev.env'),
  port: 8001,
  autoOpenBrowser: true,
  assetsSubDirectory: 'assets',
  assetsPublicPath: '/',
  notifyOnErrors: true,
  proxyTable: {
    '/v1': {
      target: 'http://dev.example.com/api/v1', // 设置调用接口域名和端口号别忘了加http
      changeOrigin: true,
      pathRewrite: {
        '^/v1': '/' // 这里理解成用'/v1'代替target里面的地址，组件中我们调接口时直接用v1代替
      }
    }
  }
},
```

样式处理

使用 sass 作为 css 预处理器，相比传统 css，sass 支持 mixins、参数混入、样式嵌套、运算、颜色功能、命名空间、作用域等，完全兼容 css3。

```
nav {  
  ul {  
    margin: 0;  
    padding: 0;  
    list-style: none;  
  }  
  
  li { display: inline-block; }  
  
  a {  
    display: block;  
    padding: 6px 12px;  
    text-decoration: none;  
  }  
}
```

数据模拟

使用 Mockjs 作为数据模拟请求，进而在开发阶段脱离后端接口并行开发。

```
Mock.mock( url, post/get , 返回的数据);
Mock.mock(/v1\/login\/operatorLogin/, 'post', {
  //登录
  ret: 0,
  data: {
    optorCode: Random.id(),
    name: Random.cname(), // Random.cname() 随机生成一个常见的中文姓名
    createTime: Random.date() + ' ' + Random.time(), // Random.date() 指示生成的日期字符串的格式
    uid: Random.guid(),
    id: Random.id(),
    token: Random.guid()
  }
});
```

/05

编码规范



CSS 命名规范 - BEM

BEM的意思就是块（block）、元素（element）、修饰符（modifier），是由[Yandex](#)团队提出的一种前端命名方法论。这种巧妙的命名方法让你的CSS类对其他开发者来说更加透明而且更有意义。BEM命名约定更加严格，而且包含更多的信息，它们用于一个团队开发一个耗时的大项目。

```
.block{}  
.block__element{}  
.block--modifier{}
```

```
.el-form-item {  
  .el-form-item__label {  
    width: 100px;  
  }  
  .el-form-item__content {  
    margin-left: 0;  
    .el-form-item__error {  
      margin-left: 100px;  
    }  
  }  
}
```

组件编写规范

- css 编写遵循 BEM 思想
- 公共组件以短横线 (kebab-case) 命名
- 公共组件样式编写严禁使用 scoped (给外部以重写的机会)
- 页面级组件建议使用 scoped 或 父级样式做样式作用域隔离，避免污染全局样式
- 页面级组件使用驼峰命名，引用公共组件使用短横线

JavaScript

- 变量使用驼峰命名
- 常量使用大写字母+短横线命名
- 关键代码强制编写注释
- 使用 ES6 语法
- Eslint 作为编码规范校验工具，提供运行时检查、代码提交前置检查

```
rules: {  
  // allow paren-less arrow functions  
  'arrow-parens': 0,  
  // allow async-await  
  'generator-star-spacing': 0,  
  // 禁止直接使用 Object.prototype 的内置属性  
  'no-prototype-builtins': 0,  
  // allow debugger during development  
  'no-debugger': process.env.NODE_ENV === 'production' ? 2 : 0,  
  // 强制使用有效的 JSDoc 注释  
  'valid-jsdoc': 0,  
  'space-before-function-paren': 0,  
  'no-inner-declarations': 0,  
  'no-regex-spaces': 2, // 禁止在正则表达式字面量中使用多个空格 /foo bar/  
  'no-proto': 2, // 禁止使用__proto__属性  
  'no-return-assign': 1, // return 语句中不能有赋值表达式  
  'no-shadow-restricted-names': 2, // 严格模式中规定的限制标识符不能作为声明时的变量名使用  
  'no-self-compare': 2, // 不能比较自身  
  'no-this-before-super': 0, // 在调用super()之前不能使用this或super  
  'no-extend-native': 0, // 可以使用扩展方法  
  'no-const-assign': 2, // 禁止修改const声明的变量  
  'no-constant-condition': 2, // 禁止在条件中使用常量表达式 if(true) if(1)  
  'no-dupe-keys': 2, // 在创建对象字面量时不允许键重复 {a:1,a:1}  
  'no-dupe-args': 2, // 函数参数不能重复  
  'no-duplicate-case': 2, // switch中的case标签不能重复
```

/05

构建部署



Webpack 配置

- 配置输入输出目录

```
output: {  
  path: config.build.assetsRoot,  
  filename: '[name].js',  
  publicPath: envConfig.assetsPublicPath  
},
```

- 配置资源别名

```
alias: {  
  vue$: 'vue/dist/vue.esm.js',  
  '@': resolve('src'),  
  assets: path.resolve(var __dirname: string ets'),  
  styles: path.resolve(__dirname, '../src/css'),  
  plugins: path.resolve(__dirname, '../src/plugins'),  
  store: path.resolve(__dirname, '../src/store'),  
  router: path.resolve(__dirname, '../src/router'),  
  static: path.resolve(__dirname, '../static')  
}
```

Webpack 配置

- 配置资源处理器

```
{
  test: /\.js$/,
  loader: 'happypack/loader?id=happyBabel',
  include: [
    resolve('src'),
    resolve('test'),
    resolve('node_modules/vehicle-keyboard/src')
  ]
},
{
  test: /\.?(png|jpe?g|gif|svg)(\?.*)?$/,
  loader: 'url-loader',
  options: {
    limit: 10000,
    name: utils.assetsPath('img/[name].[hash:7].[ext]')
  }
},
{
  test: /\.?(mp4|webm|ogg|mp3|wav|flac|aac)(\?.*)?$/,
  loader: 'url-loader',
  options: {
    limit: 10000,
    name: utils.assetsPath('media/[name].[hash:7].[ext]')
  }
},
{
  test: /\.?(woff2?|eot|ttf|otf)(\?.*)?$/,
  loader: 'url-loader',
  options: {
    limit: 10000,
    name: utils.assetsPath('fonts/[name].[hash:7].[ext]')
  }
}
```

开发阶段

Nodejs 搭配 Http 代理中间件、Webpack 开启本地微服务，代理转发 Restful API 解决
跨越问题，至此热重载

```
// handle fallback for HTML5 history API
app.use(require('connect-history-api-fallback')());

// serve webpack bundle output
app.use(devMiddleware);

// enable hot-reload and state-preserving
// compilation error display
app.use(hotMiddleware);

// serve pure static assets
let staticPath = path.posix.join(
  config.dev.assetsPublicPath,
  config.dev.assetsSubDirectory
);
app.use(staticPath, express.static('./static'));
const IP = require('ip');
const ip = IP.address();
// let uri = 'http://localhost:' + port;
let uri = 'http://' + ip + ':' + port;
let _resolve;
let readyPromise = new Promise(resolve => {
  _resolve = resolve;
});

// console.log('> Starting dev server...')
devMiddleware.waitUntilValid(() => {
  console.log('> Listening at ' + uri + '\n');
  // when env is testing, don't need open it
  if (autoOpenBrowser && process.env.NODE_ENV !== 'testing') {
    opn(uri);
  }
  _resolve();
});
```

构建 – 代码压缩

压缩 js 文件的大小，能加快网络传输，减少服务器的流量负担，也能起到混淆代码的作用。

```
new UglifyJSPlugin({
  cache: true,
  parallel: true, //允许并发
  uglifyOptions: {
    compress: {
      warnings: false,
      //可以兼容ie浏览器
      drop_console: !config.build.productionSourceMap,
      // 内嵌定义了但是只用到一次的变量
      collapse_vars: true,
      // 提取出出现多次但是没有定义成变量去引用的静态值
      reduce_vars: true
    },
    output: {
      beautify: false,
      comments: false // remove all comments,
    }
  },
  sourceMap: config.build.productionSourceMap
}),
```

构建 – CSS分割

随着项目迭代进行，页面所需要的 css 越来越多，css 文件越来越大，在今天追求性能体验的前提下，如何减少 http 请求数、减少文件大小成为了新的方向，将原有大 css 拆分成若干个业务子模块 css，按需加载

```
// extract css into its own file
new ExtractTextPlugin({
  filename: utils.assetsPath('css/[name].[contenthash].css')
}),
// Compress extracted CSS. We are using this plugin so that possible
// duplicated CSS from different components can be deduped.
new OptimizeCSSPlugin({
  cssProcessorOptions: {
    safe: true
  }
}),
new CSSSplitWebpackPlugin({
  size: 4000,
  filename: utils.assetsPath('css/[name]-[part].[ext]')
}),
```

构建 – 业务分离

模块化的开发方式解放了前端，但也带来了新的问题，打包过后 js 文件过大，业务代码和第三方类库揉在一起，造成加载缓慢，严重影响用户体验，基础库与业务的分离势在必行。

- 将模块化引入的类库统一打包到 vendor.js 文件

```
new webpack.optimize.CommonsChunkPlugin({
  name: 'vendor',
  minChunks: ({ resource }) =>
    resource &&
    resource.indexOf('node_modules') >= 0 &&
    resource.match(/\.js$/)
}),|
```

构建 – 提取公共模块

- 提取出现两次以上的类库

```
new webpack.optimize.CommonsChunkPlugin({  
  async: 'used-twice',  
  minChunks: (module, count) => count >= 2  
}),
```

- 提取异步模块中引用的公共类库

```
new webpack.optimize.CommonsChunkPlugin({  
  async: 'common-in-lazy',  
  minChunks: ({ resource } = {}) =>  
    resource && resource.includes('node_modules') && /axios/.test(resource)  
}),
```

构建 – 文件指纹

为了提升页面加载速度，缓存是必不可少的环节，浏览器会缓存加载的文件，每次加载先从本地缓存中查找，命中缓存直接返回。

每次构建针对文件内容做 md5 编码提取 hash 值，文件无变化则 hash 值相同，重新加载浏览器只会重新加载 hash 值变化的文件

```
filename: utils.assetsPath('js/[name].[chunkhash].js'),  
chunkFilename: utils.assetsPath('js/[id].[chunkhash].js'),
```

```
new ExtractTextPlugin({  
  filename: utils.assetsPath('css/[name].[contenthash].css')  
}),
```

构建 – 打包

配置项目所需资源处理器，配置输出目录，执行 Webpack 构建

```
const ora = require('ora');
const rm = require('rimraf');
const path = require('path');
const chalk = require('chalk');
const webpack = require('webpack');
const config = require('../config');
const webpackConfig = require('../webpack.prod.conf');
const env = config.build[process.env.ENV_CONFIG + 'Env'];
const emitServer =
  process.env.emit_server === true || process.env.emit_server === 'true';
var fs = require('fs');

const spinner = ora('building for production...');
spinner.start();

rm(path.join(config.build.assetsRoot, config.build.assetsSubDirectory), err => {
  if (err) throw err;

  var SERVER = env.SERVER;
  if (emitServer) {
    if (!SERVER) throw new Error('Server Config is not exists');
    if (!SERVER.DIR) throw new Error('Server.Dir is not exists');

    let existsSync = fs.existsSync(SERVER.PRIVATE_KEY);
    if (!existsSync) throw new Error('Server.PrivateKey is not exists');
  }
  webpack(webpackConfig, function(err, stats) {...
```

构建 – 部署

Webpack 打包完毕，通过 nodejs ssh 自动上传目标服务器

```
var conn = new (require('ssh2')).Client(),
    localPath = `${SERVER.DIR}/dist.zip`,
    remotePath = SERVER.DIR;
conn
  .on('ready', function() {
    conn.exec(
      `mv ${remotePath}/dist.zip ${remotePath}/dist.zip.bak`,
      function(err, stream) {
        if (err) throw err;
        conn.sftp(function(err, sftp) {
          if (err) throw err;
          const loading = ora('Uploading...');
          sftp.fastPut(
            path.resolve(__dirname, '../dist/dist.zip'),
            localPath,
            {
              step(total_transferred, chunk, total) {
                if (total_transferred < total) {
                  loading.start();
                }
              }
            },
            function(err) { ...
          }
        );
      }
    );
  })
  .connect({
    host: SERVER.HOST,
    port: SERVER.PORT,
    username: SERVER.USER_NAME,
    privateKey: fs.readFileSync(SERVER.PRIVATE_KEY)
  });
```

构建 – 命令

- `npm run dev` : 开发环境热更新
- `npm run build:sit` : 构建 SIT 环境部署包 (仅构建)
- `npm run build:sit:server` : 构建 SIT 并部署 SIT 环境
- `npm run build:uat` : 构建 UAT 环境部署包 (仅构建)
- `npm run build:prod` : 构建 PROD 环境部署包 (仅构建)
- `npm run build:prod:server` : 构建 PROD 并部署 PROD 环境

/07

常见问题



JavaScript 兼容问题

ES6 的标准早在09年 ES5 发布时开始准备，在15年6月份发布，是历经6年厚积薄发。增加不少新特性、新函数以及新的语法。比如新增添局部变量声明方法，常量声明方法，语句块成了局部作用域，模板字符串（新的字符串声明方式，支持直接变量），解构赋值，箭头函数（函数简写方式），新的“...”符号，继续迭代器（yield），模块，class（类）等等等等新特性。

发展至今并非所有浏览器均已支持 es6，比如 IE，为了兼容旧版浏览器，我们需要使用 Babel 把 es6 转换成 es5 代码，使用 polyfill 抚平不同浏览器之间对 js 实现的差异。

```
"presets": [  
  ["env", { "modules": false }],  
  ["es2015", { "loose": true, "modules": false }],  
  "stage-2"  
],
```

CSS 兼容问题

浏览器的兼容性问题，通常是因为不同的浏览器对同一段代码有不同的解析，造成页面显示不统一的情况。对于某些支持受限的属性，针对不同的浏览器添加相应的前缀，比如-webkit-、-o-、-moz-。

引入 reset.css 解决各浏览器默认属性差异，通过 postcss 自动为 css 添加浏览器兼容前缀

```
module.exports = {  
  from: undefined,  
  plugins: {  
    // to edit target browsers: use "browserlist"  
    autoprefixer: {},  
    'postcss-flexibility': {}  
  }  
};
```

```
"browserslist": [  
  "defaults",  
  "> 1%",  
  "ie >= 9"  
]
```

业务分离后包体积过大

前面提到分离业务代码和第三方类库代码，生成 vendor.js 文件，但是随着项目的复杂，第三方类库的大小和数量还是变得不可控，不可避免的造成 vendor.js 的增大，即使开启了 Gzip，依然收效甚微。

通过配置非打包类库，提取第三方库使用 CDN 加载，打包仅打包非过大尺寸类库

```
},
externals: {
  vue: 'Vue',
  'element-ui': 'ELEMENT',
  xlsx: 'XLSX',
  'v-charts': 'VeIndex'
},
```

```
<!-- 引入组件库 -->
<script src="https://cdn.jsdelivr.net/npm/element-ui@2.4.7/lib/index.js"></script>
<script src="https://cdn.jsdelivr.net/npm/xlsx/jszip.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/xlsx/xlsx.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/echarts/dist/echarts.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/v-charts/lib/index.min.js"></script>
```

区分构建环境

项目构建不可避免的需要区分运行环境，连接不同的 API 服务，比如 DEV、SIT、UAT、PROD。通过配置不同的构建命令，执行不同的环境构建脚本，灵活打包

```
"scripts": {  
  "dev": "cross-env NODE_ENV=development ENV_CONFIG=dev node build/dev-server.js",  
  "build:sit": "cross-env NODE_ENV=production ENV_CONFIG=sit node build/build.js",  
  "build:sit:server": "cross-env NODE_ENV=production ENV_CONFIG=sit emit_server=true node build/build.js",  
  "build:uat": "cross-env NODE_ENV=production ENV_CONFIG=uat node build/build.js",  
  "build:prod": "cross-env NODE_ENV=production ENV_CONFIG=prod node build/build.js",  
  "build:prod:server": "cross-env NODE_ENV=production ENV_CONFIG=prod emit_server=true node build/build.js"  
},
```

结语

前端工程化体系的引入，让前端开发能和原生App应用项目开发一样“自成体系”，脱离了对后端项目的依赖。基于“约定优于配置”、“按照约定写代码”的原则对Node层功能的设定能够降低沟通协调成本，构建、部署等工作的规范化，使前端技术人员的开发重点回归到Web应用的交互体验本身，回归到“纯粹”的前端研发。



谢谢

前端开发组
xxxx有限公司